

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	Ruthika Reddy C	Reg. No.:	192421138
Section / Batch:	BTECH IT	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Debugging String Matching Code (Incorrect Match Detection Condition)

The following brute force string matching implementation breaks correctly on mismatches, but the condition used to declare a full match is logically unsafe and depends on the final value of the loop variable. Carefully analyze why this may lead to incorrect behavior in some cases. Apply debugging so that a match is reported only when every character in the pattern has been successfully compared. Optimize the implementation for clarity while retaining brute force logic. Analyze the time complexity and rewrite the final corrected code with comments.

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    result = []
    for i in range(n - m + 1):
        for j in range(m):
            if text[i+j] != pattern[j]:
                break
        if j == m-1:
            result.append(i)
    return result
```

A) Problem Analysis

The given brute force string matching algorithm compares the pattern with every possible position in the text. It correctly stops (break) when a mismatch occurs. However, the condition:

if $j == m - 1$:

is logically unsafe because it only checks the final value of j .

Why is it incorrect?

- When a mismatch occurs, the loop breaks early, but j still keeps its last value.
- The condition $j == m - 1$ does not guarantee that all characters matched.
- A match should be reported only if every character in the pattern has been successfully compared.

Corrected Code

```
def string_match(text, pattern):
```

```
    n = len(text)
```

```
    m = len(pattern)
```

```
    result = []
```

```
    for i in range(n - m + 1):
```

```
        for j in range(m):
```

```
            if text[i + j] != pattern[j]:
```

```
                break
```

```
        else:
```

```
            result.append(i)
```

```
    return result
```

```
text = "ABABCABCAB"
```

```
pattern = "ABC"
```

```
print("Pattern found at positions:", string_match(text, pattern))
```

Comments

- The for...else block executes only when no break occurs.
- This ensures every character in the pattern matches before storing the index.
- The brute force approach remains unchanged but becomes more reliable.

Time Complexity

- Best Case: $O(n)$
- Worst Case: $O(n \times m)$
- Space Complexity: $O(k)$, where k is the number of matching positions.

Sample Output

Pattern found at positions: [2, 5]

Conclusion

The corrected code reports a match only after all characters in the pattern are successfully compared, making the brute force string matching algorithm accurate and reliable.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____